Software Engineering-2

Smart Rental Application

Table of Contents

1 Software Requirements Specification

2 Use Case Diagram

3 Use Case Diagram

4 Class Diagram (User service)

5 ERD (User service)

6 Activity Diagram (User service)

7 Sequence Diagram (User service)

8 Class Diagram (Property service)

9 ERD (Property service)

10 Activity Diagram (Property service)

11 Sequence Diagram (Property service)

12 Class Diagram (Visit service)

13 ERD (Visit service)

14 Activity Diagram (Visit service)

15 Sequence Diagram (Visit service)

16 Class Diagram (Rental service)

17 ERD (Rental service)

18 Activity Diagram (Rental service)

19 Sequence Diagram (Rental service)

All Diagrams Online Link:
https://drive.google.com/file/d/1dlB3O145IijzZkEG5c_k2j_cU2FSEXLF/view?usp=drive_link

SRS

1. Introduction

1.1 Purpose

The primary purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive description of the functional and technical requirements for the SmartRent property management platform. It serves as a definitive guide for developers, testers, and stakeholders to ensure that the system is built according to the specified business logic and microservices architecture.

1.2 Scope

SmartRent is a microservices-based application designed to streamline the rental process for both tenants and landlords.

- **Tenants:** The platform allows users to browse available properties, apply advanced search filters, schedule property visits, and submit formal rental applications including document uploads.

- **Landlords:** The system enables property owners to list their properties, manage high-quality image uploads, and process visit requests or rental applications (approve/reject/reschedule).

- **Admins:** The platform provides a centralized dashboard to moderate users, approve property listings, and monitor platform-wide statistics. The system is built using Spring Boot for the backend, React for the frontend, and SQL Server for data persistence across four isolated databases.

1.3 Definitions, Acronyms, and Abbreviations

To ensure clarity throughout the document, the following technical terms are defined:

- **JWT (JSON Web Token):** A compact, URL-safe means of representing claims to be transferred between two parties, used for stateless authentication in this project.

- **Microservices:** An architectural style that structures an application as a collection of small, autonomous services modeled around a business domain.

- **Feign Client:** A declarative web service client used to simplify communication between microservices (ex: between the Visit Service and Property Service).

- **SQL Server:** A relational database management system used to host the four independent databases: users_db, properties_db, visits_db, and rentals_db.

- **SRS:** Software Requirements Specification document.

- **RBAC (Role-Based Access Control):** A method of restricting system access to authorized users based on their roles (Tenant, Landlord, Admin).

2. Overall Description

2.1 System Perspective

The system is composed of four primary microservices communicating over HTTP via OpenFeign. Security is centralized at the API Gateway, which validates JWTs and injects X-UserId and X-User-Role headers into downstream requests.

2.2 User Classes and Characteristics

- Tenant: Can search/filter properties, manage a "Favorites" list, book visits, and submit rental applications with supporting documents.
- Landlord: Responsible for listing properties, managing visit requests (approve/reject/reschedule), and reviewing tenant applications.
- Admin: A high-level actor who moderates the platform by approving properties and managing user account statuses.

2.3 Design and Implementation Constraints

- **Database Isolation:** No cross-database SQL joins are permitted; data consistency is maintained via service-to-service calls.
- **Statelessness:** The backend must remain stateless, relying on JWTs for session management.
- **Environment:** Development is standardized on Java 21, Spring Boot 3.2.5, and Docker Compose.

SRS

3. Functional Requirements

3.1 Authentication & Authorization

- **FR-01:** The system shall allow users to register as a Tenant or Landlord with specific status assignments (ex: Landlord as PENDING).
- **FR-02:** The system shall authenticate users via email and password, generating a stateless JWT upon successful login.
- **FR-03:** The system shall assign roles (Tenant, Landlord, Admin) and include these roles in the JWT claims.
- **FR-04:** The system shall perform status checks to ensure only ACTIVE users can access protected services.
- **FR-05:** The system shall allow users to view and update their own profiles via the User Service.
- **FR-06:** The system shall provide an internal endpoint for other services to retrieve user details via Feign Client.

3.2 Property Management

- **FR-07:** Landlords shall be able to perform CRUD operations (Create, Read, Update, Delete) on their property listings.
- **FR-08:** The system shall support multi-image uploads (JPEG/PNG) and designate one as the main image.
- **FR-09:** Tenants shall be able to search properties using advanced filters such as city, rent range, and amenities.
- **FR-10:** Properties shall remain in a PENDING status until approved by an Admin.

3.3 Visit & Rental Management

- **FR-11:** Tenants shall be able to book visits, while the system performs conflict checks to prevent overlapping appointments.
- **FR-12:** Landlords shall have the authority to approve, reject, or reschedule visit requests.
- **FR-13:** Tenants shall be able to submit rental applications and upload necessary documentation (PDF/Images).
- **FR-14:** The system shall capture a snapshot of property details (title, rent) at the time of application to maintain record integrity

3.4 Reviews, Favorites & Admin Tools

- **FR-15:** Tenants shall be able to write and manage reviews for approved properties (limited to one review per tenant per property).
- **FR-16:** Admins shall be able to list all users, update user statuses, and view overall platform statistics.

4. Non-Functional Requirements

4.1 Security

- The system shall use BCrypt for password hashing and enforce a 24-hour expiry for all JWT tokens.
- All communications in production environments shall be secured via HTTPS.

4.2 Performance

- Property search queries and filtering results shall be returned to the user in less than 2 seconds.

4.3 Scalability & Availability

- Each microservice shall be containerized using Docker, allowing it to be independently deployable and scalable.
- The system shall implement Docker health checks and automatic Eureka re-registration to ensure high availability.

4.4 Maintainability

- The system shall utilize AOP (Aspect-Oriented Programming) for centralized logging and Swagger/OpenAPI for automated API documentation.

Diagrams

1. Use Case Diagram

The Use Case Diagram for the SmartRent application illustrates the interactions between four primary actors: the Tenant, Landlord, Admin, and the Gateway. The system employs an inheritance structure where both Tenants and Landlords inherit basic functionalities from an Abstract User, such as registration, login, and profile management. The Tenant focus is on property discovery and acquisition, including browsing properties, booking visits, and submitting rental applications, while the Landlord is responsible for property management tasks like uploading images and managing visit or rental requests.

Strategically, the diagram highlights critical system dependencies: activities such as booking a visit or submitting an application include viewing property details, and all protected actions are linked to a Validate JWT use case handled by the Gateway to ensure secure, stateless communication. Finally, the Admin actor serves as a moderator, overseeing the platform by approving landlord accounts and property listings, and monitoring performance through platform statistics.

Diagrams

2. ERD (All System)

The provided image presents a comprehensive Entity-Relationship Diagram (ERD) for the SmartRent system, integrating the various microservices into a unified data model. This centralized view illustrates how users, properties, visits, and rental applications interact across the platform.

Core Entities and Global Relationships

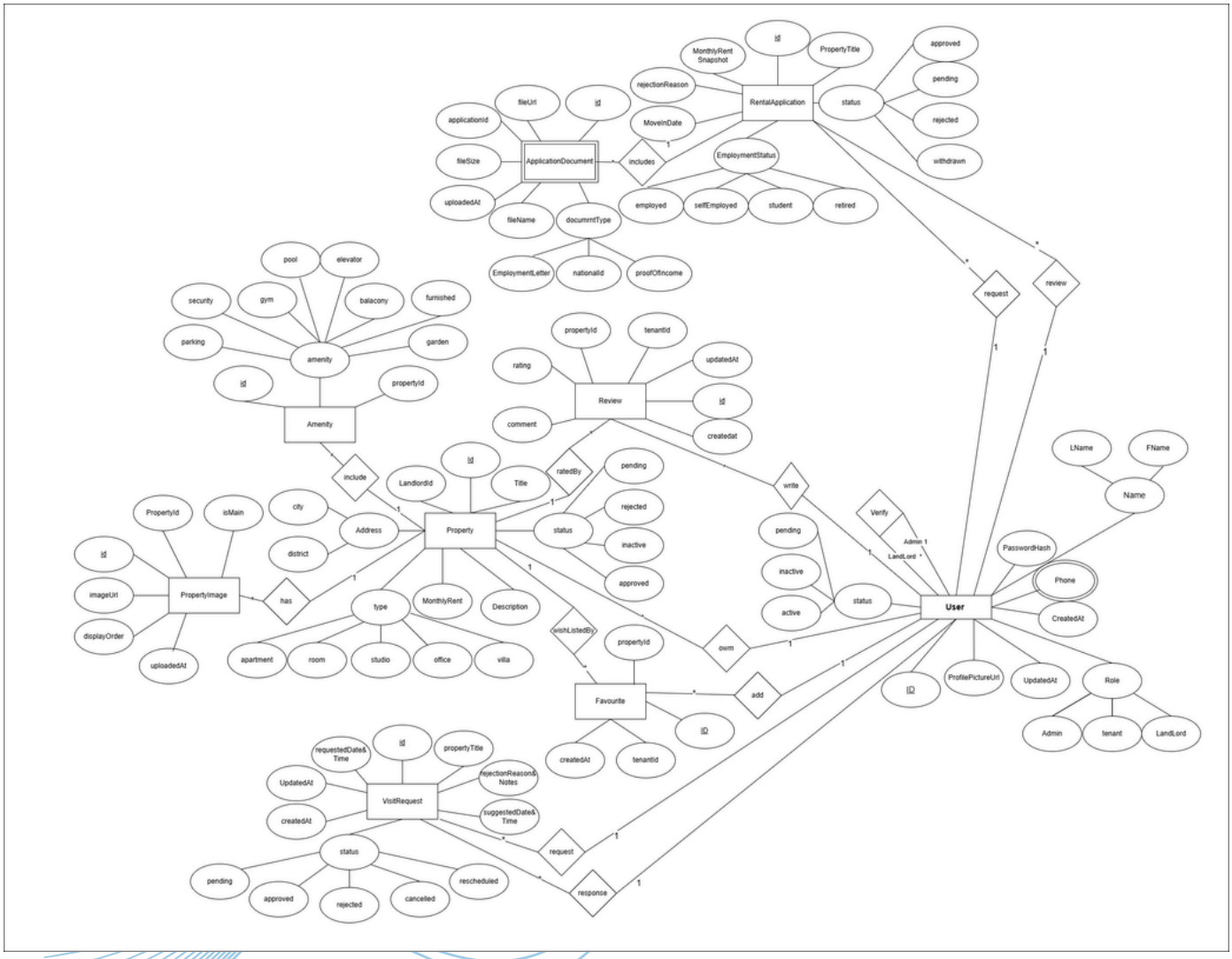
- User Entity: Serves as the primary actor, with relationships extending to all other modules. A User (Admin) can Verify multiple Landlords. Tenants (a role of User) can write reviews, add properties to their Favourites, and request both visits and rental applications.
- Property Entity: Acts as the central object of the marketplace. It is owned by a Landlord (User) and includes various Amenities and PropertyImages. Properties are the targets of Reviews, VisitRequests, and RentalApplications.

Key Service Modules

- Rental Management: A RentalApplication captures the intent of a Tenant to lease a Property. It includes a weak entity, ApplicationDocument, which stores essential verification files like proof of income or national IDs.
- Visit Management: The VisitRequest entity facilitates scheduling between the Tenant and Landlord. It includes specific logic for negotiation, such as suggestedDate&Time and rejectionReason.
- Feedback and Personalization: The Review entity allows Tenants to rate and comment on properties they have interacted with, while the Favourite entity allows for a personalized shortlist of listings.

Logical Constraints

- Status Enumerations: Every major entity (User, Property, VisitRequest, RentalApplication) features a status attribute to manage its lifecycle — ensuring, for example, that only approved properties are visible or only active users can perform transactions.
- Snapshots: To maintain historical integrity, the RentalApplication stores a MonthlyRentSnapshot, ensuring the financial terms are preserved even if the Landlord later updates the Property's rent.

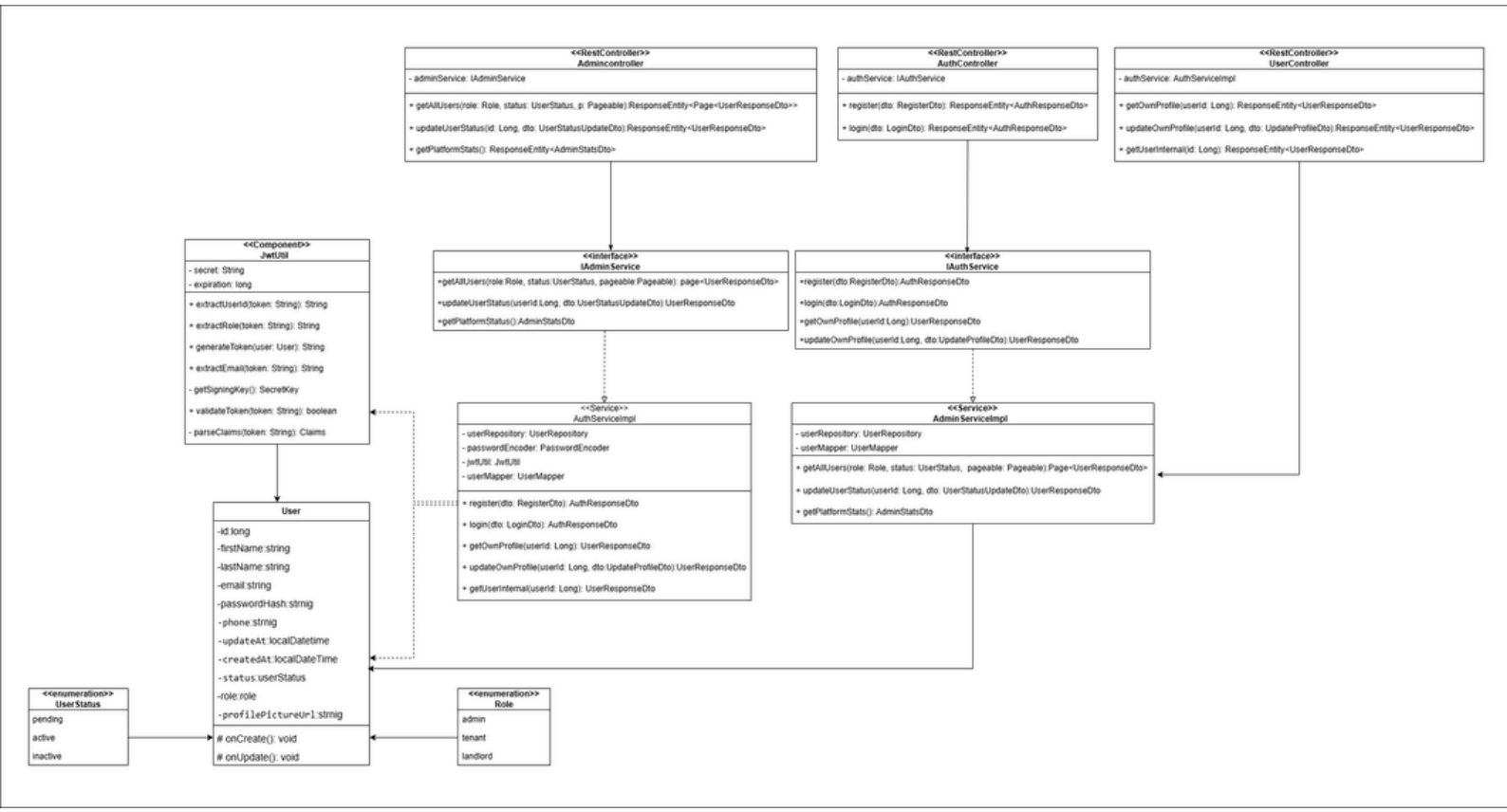


Diagrams

3-Class Diagram (User Service)

The provided Class Diagram illustrates the internal architecture of the User Service for the SmartRent platform. It follows a structured Controller-Service-Repository pattern, where AdminController, AuthController, and UserController act as the entry points for external API requests. These controllers interact with service interfaces like IAdminService and IAuthService, which are implemented by AdminServiceImpl and AuthServiceImpl to handle business logic such as user registration, login, and profile management.

The core of the data model is the User entity, which includes essential attributes like email, password hash, role, and status. The diagram also highlights a dedicated JwtUtil component responsible for security tasks, including generating and validating JWT tokens. Furthermore, the system uses enumerations for Role (Admin, Tenant, Landlord) and UserStatus (Pending, Active, Inactive) to strictly manage user permissions and account states throughout the application lifecycle.



OCL For User Service class Diagram:

1- context User
inv UniqueEmail:
User.allInstances()->isUnique(email)

2-context AuthServiceImpl::register(dto: RegisterDto) : AuthResponseDto
pre EmailMustNotExist:
userRepository.findByEmail(dto.email)->isEmpty()

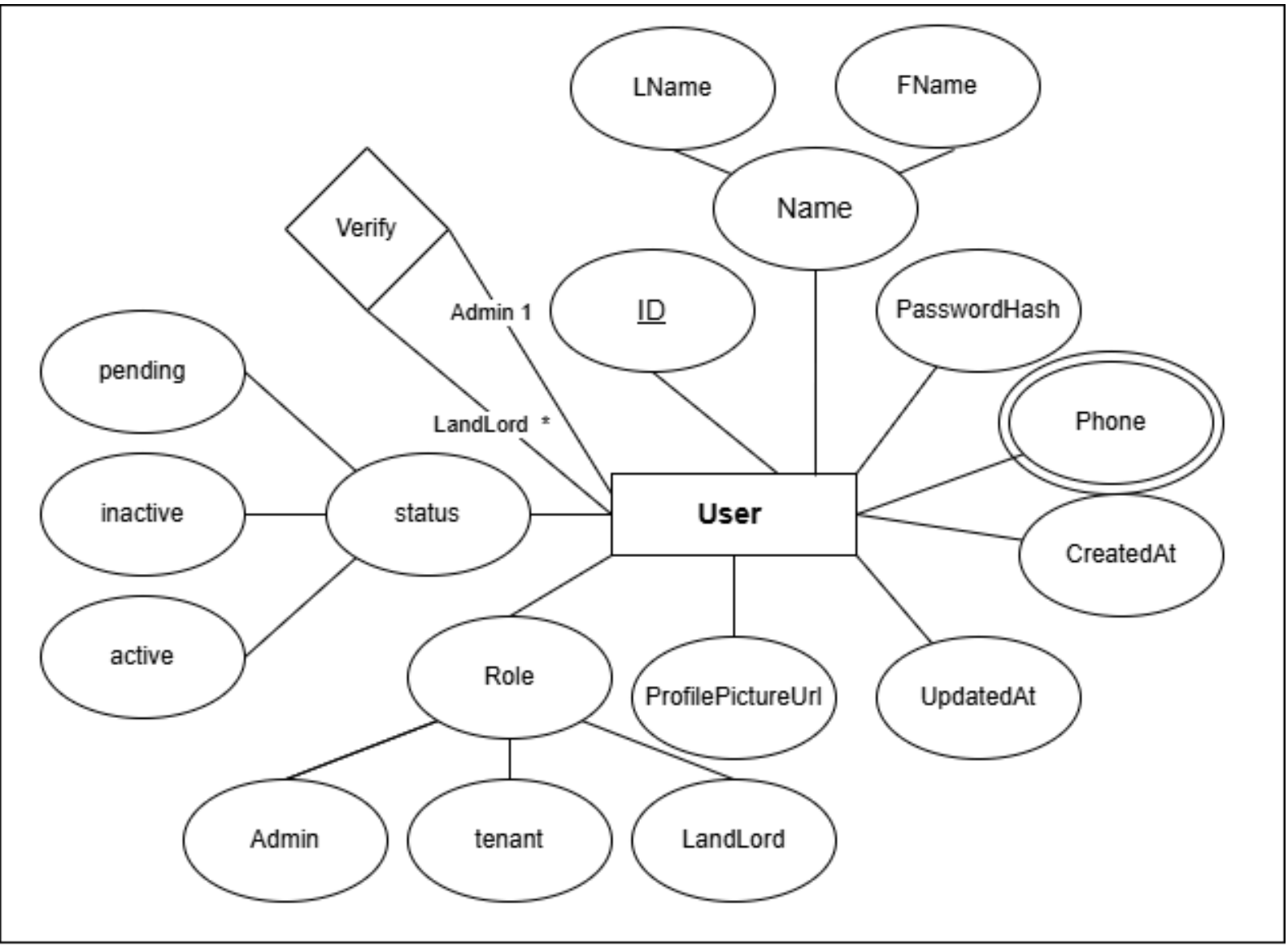
3-context AuthServiceImpl::register(dto: RegisterDto) : AuthResponseDto
post DetermineInitialStatus:
let savedUser : User = userRepository.findByEmail(dto.email)->first() in
(dto.role = Role::LANDLORD implies savedUser.status = UserStatus::PENDING) and
(dto.role <> Role::LANDLORD implies savedUser.status = UserStatus::ACTIVE)

```
4-context AuthServiceImpl::login(dto: LoginDto) : AuthResponseDto
pre ValidCredentials:
 let user : User = userRepository.findByEmail(dto.email)->first() in
 user <> null and passwordEncoder.matches(dto.password, user.passwordHash)
5-context AuthServiceImpl::login(dto: LoginDto) : AuthResponseDto
pre AccountMustBeActive:
   let user : User = userRepository.findByEmail(dto.email)->first() in
   user.status = UserStatus::ACTIVE
6-context AuthServiceImpl::updateOwnProfile(userId: Long, dto: UpdateProfileDto) : UserResponseDto
pre UserMustExist:
   userRepository.findById(userId)->notEmpty()
7-context AuthServiceImpl::updateOwnProfile(userId: Long, dto: UpdateProfileDto) : UserResponseDto
post ProfileFieldsUpdated:
   let updatedUser : User = userRepository.findById(userId)->first() in
   (dto.firstName <> null implies updatedUser.firstName = dto.firstName) and
   (dto.lastName <> null implies updatedUser.lastName = dto.lastName) and
   (dto.phone <> null implies updatedUser.phone = dto.phone) and
   (dto.profilePictureUrl <> null implies updatedUser.profilePictureUrl = dto.profilePictureUrl)
```

Diagrams

4-ERD (User Service)

The provided Entity-Relationship Diagram (ERD), which represents the data structure for the User Service in the SmartRent platform, defines the attributes and relationships of the User entity. The user is uniquely identified by an ID and includes composite attributes like Name (consisting of First Name and Last Name) and multi-valued attributes like Phone. The diagram highlights specific enumerations for user Role (Admin, Tenant, Landlord) and Status (Pending, Active, Inactive), which are critical for the system's role-based access control and account lifecycle management. Furthermore, a recursive Verify relationship is depicted, indicating that one Admin is responsible for verifying multiple Landlords to ensure platform security and trust.

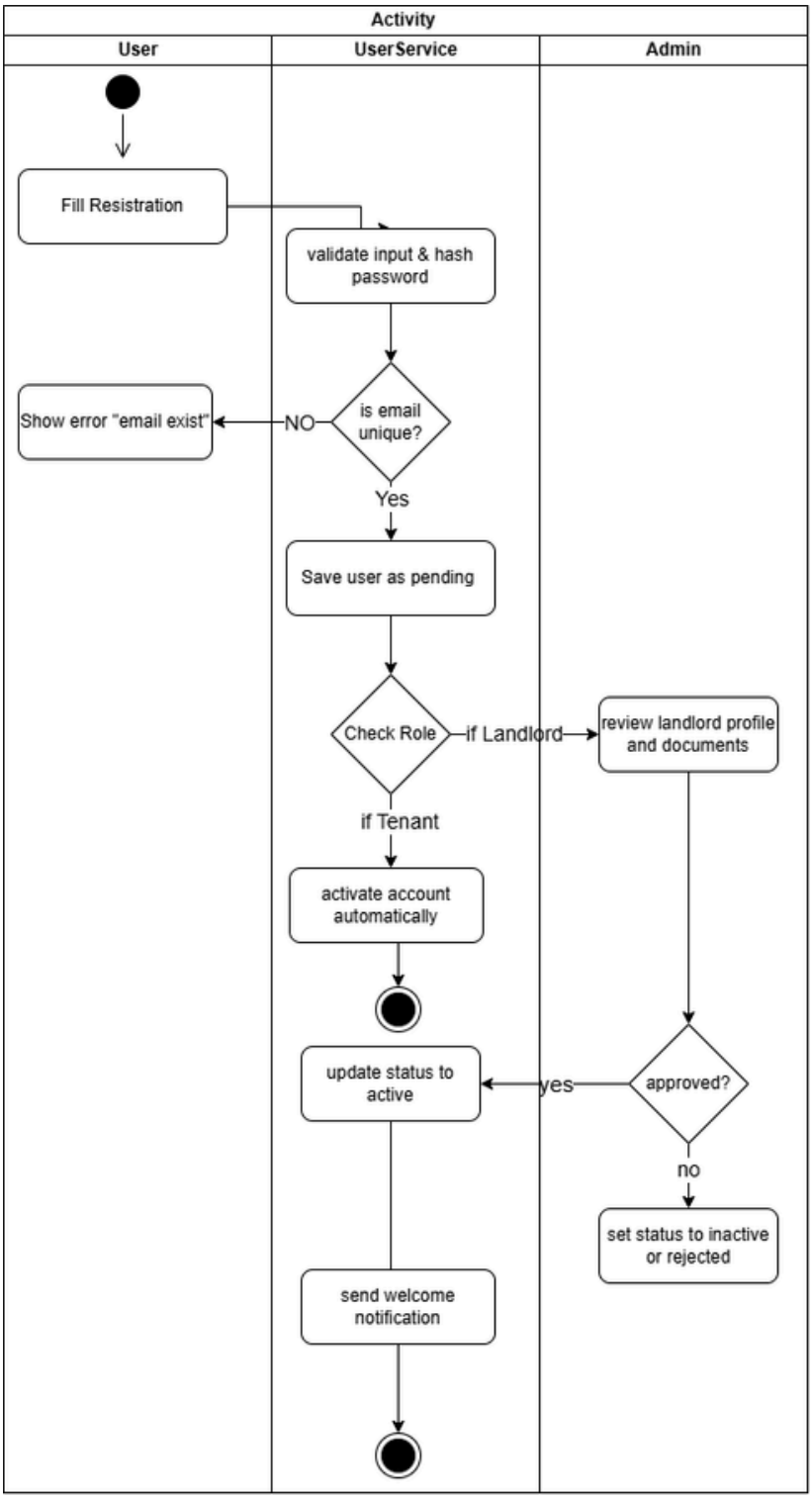


Diagrams

5-Activity Diagram (User Service)

The Activity Diagram for the User Service illustrates the comprehensive registration and account activation workflow for the SmartRent platform. The process is visualized across three lanes: the User, UserService, and Admin, starting with the user submitting their registration data. The UserService performs critical backend operations, including input validation, password hashing, and email uniqueness checks.

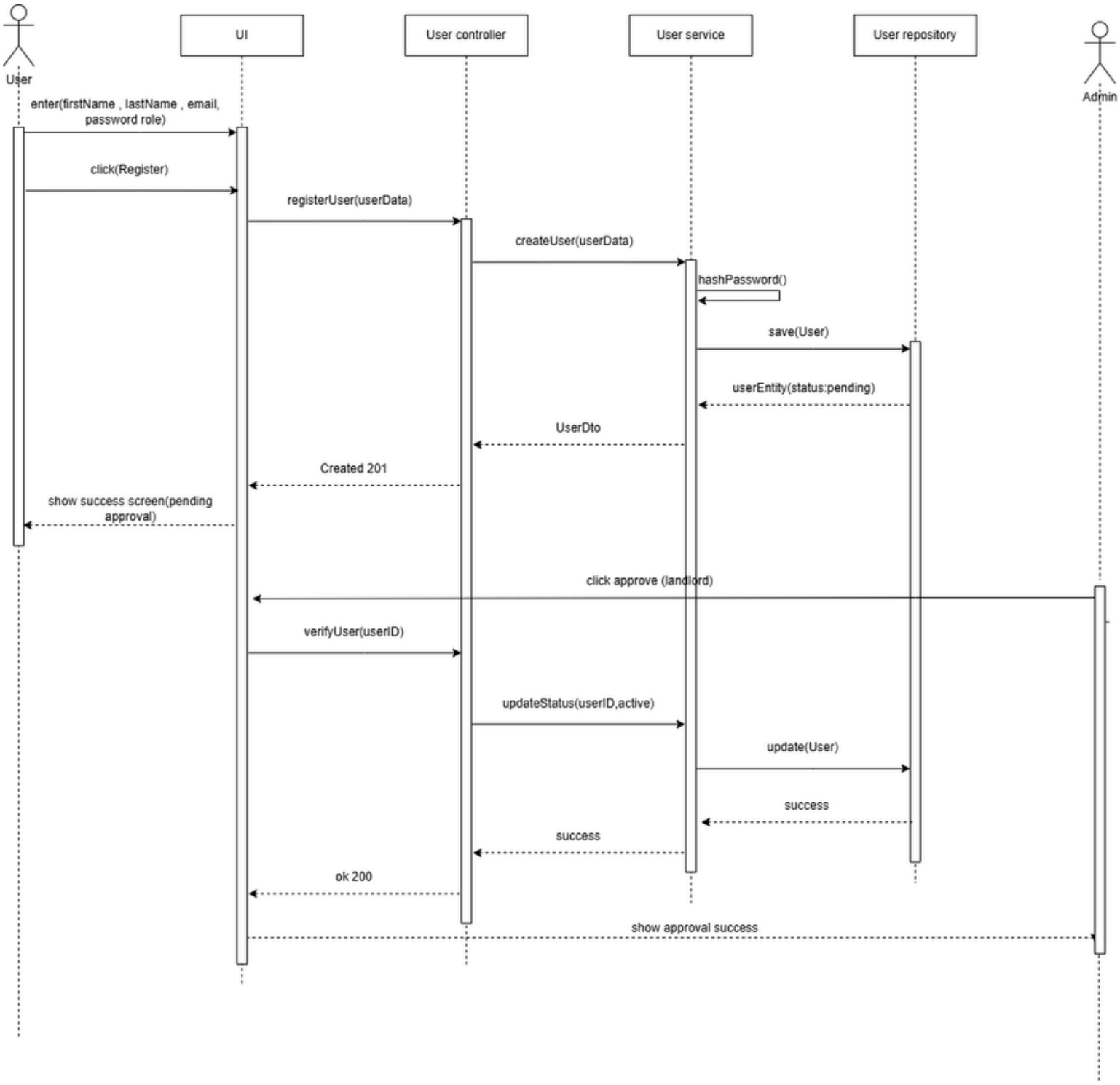
A pivotal decision point occurs based on the user's role: Tenants bypass manual review and receive automatic account activation, while Landlords are placed in a Pending state. This triggers a secondary workflow where an Admin must manually review the landlord's profile and documents. Upon admin approval, the system updates the landlord's status to Active and sends a welcome notification, whereas a rejection sets the account to Inactive or Rejected, ensuring a verified and secure environment for all platform participants.



Diagrams

6-Sequence Diagram (User Service)

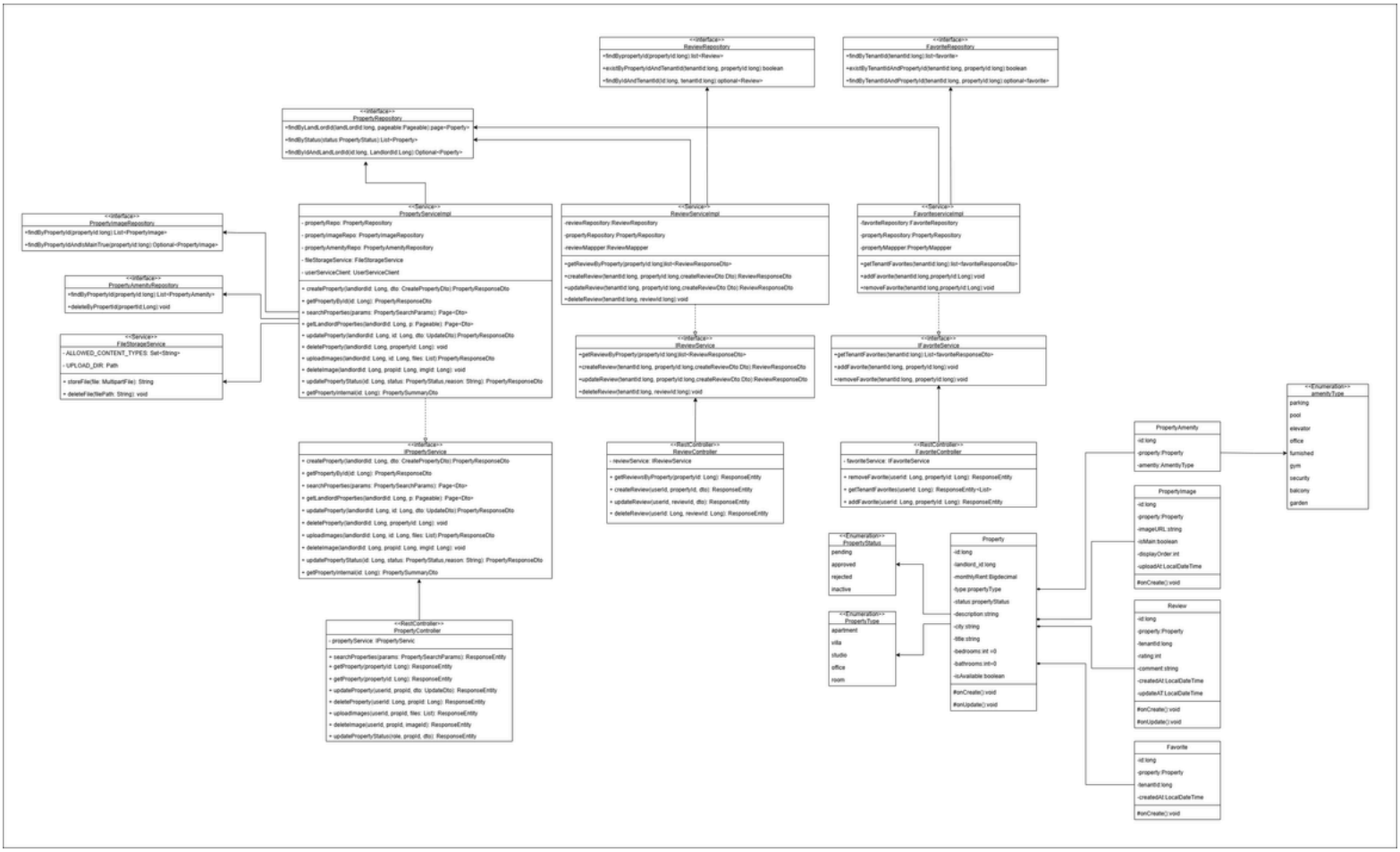
The provided Sequence Diagram depicts the technical workflow for user registration and subsequent administrator approval within the system. The interaction begins with the User submitting their details through the UI, which triggers a request to the User controller and then to the User service for password hashing and initial data persistence. During this phase, the User repository saves the entity with a pending status. Following this, the Admin intervenes by selecting to approve the user, initiating a verification request that flows back through the controller to the service. The User service then updates the status to active in the repository, resulting in a success confirmation being sent back through the layers and displayed on the UI.



Diagrams

7-Class Diagram (Property Service)

The provided Class Diagram details the technical architecture of the Property Service within the SmartRent system. It utilizes a multi-layered design pattern where PropertyController, ReviewController, and FavoriteController manage external API requests. These controllers interface with specific service layers, such as PropertyServiceImpl, which coordinates business logic across various repositories including PropertyRepository, ReviewRepository, and FavoriteRepository. The data model revolves around the Property entity, which maintains relationships with PropertyImage, PropertyAmenity, Review, and Favorite objects. To ensure consistent data handling, the diagram incorporates several enumerations, such as PropertyStatus (Pending, Approved, Rejected, Inactive) and AmenityType (Parking, Pool, Gym, etc.), to define the specific characteristics and current states of real estate listings.



OCL For Property Service class Diagram:

1-context FileStorageService::storeFile(file: MultipartFile) : String

pre ValidImageFormat:

file.contentType = 'image/jpeg' or file.contentType = 'image/png'

2-context FileStorageService::storeFile(file: MultipartFile) : String

post FilePathReturned:

result <> null and result.size() > 0

3-context FavoriteServiceImpl::addFavorite(tenantId: Long, propertyId: Long)

pre PropertyMustExist:

propertyRepository.findById(propertyId)->notEmpty()

pre NotAlreadyFavorited:

favoriteRepository.existsByTenantIdAndPropertyId(tenantId, propertyId) = false

4-context FavoriteServiceImpl::addFavorite(tenantId: Long, propertyId: Long)

post FavoriteAdded:

favoriteRepository.findByTenantId(tenantId)->size() =

favoriteRepository.findByTenantId(tenantId)@pre->size() + 1

```
5-context FavoriteServiceImpl::removeFavorite(tenantId: Long, propertyId: Long)
pre MustBeFavorited:
  favoriteRepository.existsByTenantIdAndPropertyId(tenantId, propertyId) = true
post FavoriteRemoved:
  favoriteRepository.existsByTenantIdAndPropertyId(tenantId, propertyId) = false
6-context PropertyServiceImpl::createProperty(landlordId: Long, dto: CreatePropertyDto) : PropertyResponseDto
pre LandlordMustBeActive:
  userServiceClient.getUserById(landlordId).status.equalsIgnoreCase('ACTIVE')
7-context PropertyServiceImpl::createProperty(landlordId: Long, dto: CreatePropertyDto) : PropertyResponseDto
post InitialPropertyState:
  let savedProperty : Property = propertyRepository.findByLandlordId(landlordId)->last() in
  savedProperty.status = PropertyStatus::PENDING and
  savedProperty.isAvailable = true and
  savedProperty.landlordId = landlordId
8-context PropertyServiceImpl::updateProperty(landlordId: Long, propertyId: Long, dto: UpdatePropertyDto) : PropertyResponseDto
pre MustOwnProperty:
  let property : Property = propertyRepository.findById(propertyId)->first() in
  property <> null and property.landlordId = landlordId
9-context PropertyServiceImpl::updatePropertyStatus(propertyId: Long, status: PropertyStatus, rejectionReason: String) :
PropertyResponseDto
post RejectionMakesUnavailable:
  let updatedProperty : Property = propertyRepository.findById(propertyId)->first() in
  (status = PropertyStatus::REJECTED implies updatedProperty.isAvailable = false) and
  updatedProperty.status = status
10-context PropertyServiceImpl::searchProperties(params: PropertySearchParams) : Page<PropertyResponseDto>
post OnlyApprovedAndAvailable:
  result.content->forAll(p | p.status = 'APPROVED' and p.isAvailable = true)
11-context PropertyServiceImpl::updatePropertyStatus(propertyId: Long, status: PropertyStatus, rejectionReason: String) :
PropertyResponseDto
post RejectionMakesUnavailable:
  let updatedProperty : Property = propertyRepository.findById(propertyId)->first() in
  (status = PropertyStatus::REJECTED implies updatedProperty.isAvailable = false) and
  updatedProperty.status = status
12-context PropertyServiceImpl::searchProperties(params: PropertySearchParams) : Page<PropertyResponseDto>
post OnlyApprovedAndAvailable:
  result.content->forAll(p | p.status = 'APPROVED' and p.isAvailable = true)
13-context ReviewServiceImpl::createReview(tenantId: Long, propertyId: Long, dto: CreateReviewDto) : ReviewResponseDto
pre PropertyMustExist:
  propertyRepository.findById(propertyId)->notEmpty()
pre ValidRatingRange:
  dto.rating >= 1 and dto.rating <= 5
14-context ReviewServiceImpl::createReview(tenantId: Long, propertyId: Long, dto: CreateReviewDto) : ReviewResponseDto
pre OnlyOneReviewPerTenant:
  reviewRepository.existsByPropertyIdAndTenantId(propertyId, tenantId) = false
15-context ReviewServiceImpl::updateReview(tenantId: Long, reviewId: Long, dto: CreateReviewDto) : ReviewResponseDto
pre MustOwnReviewToUpdate:
  let review : Review = reviewRepository.findById(reviewId)->first() in
  review <> null and review.tenantId = tenantId
16-context ReviewServiceImpl::deleteReview(tenantId: Long, reviewId: Long)
pre MustOwnReviewToDelete:
  let review : Review = reviewRepository.findById(reviewId)->first() in
  review <> null and review.tenantId = tenantId
17-context ReviewServiceImpl::deleteReview(tenantId: Long, reviewId: Long)
post ReviewIsDeleted:
  reviewRepository.findById(reviewId)->isEmpty()
```

Diagrams

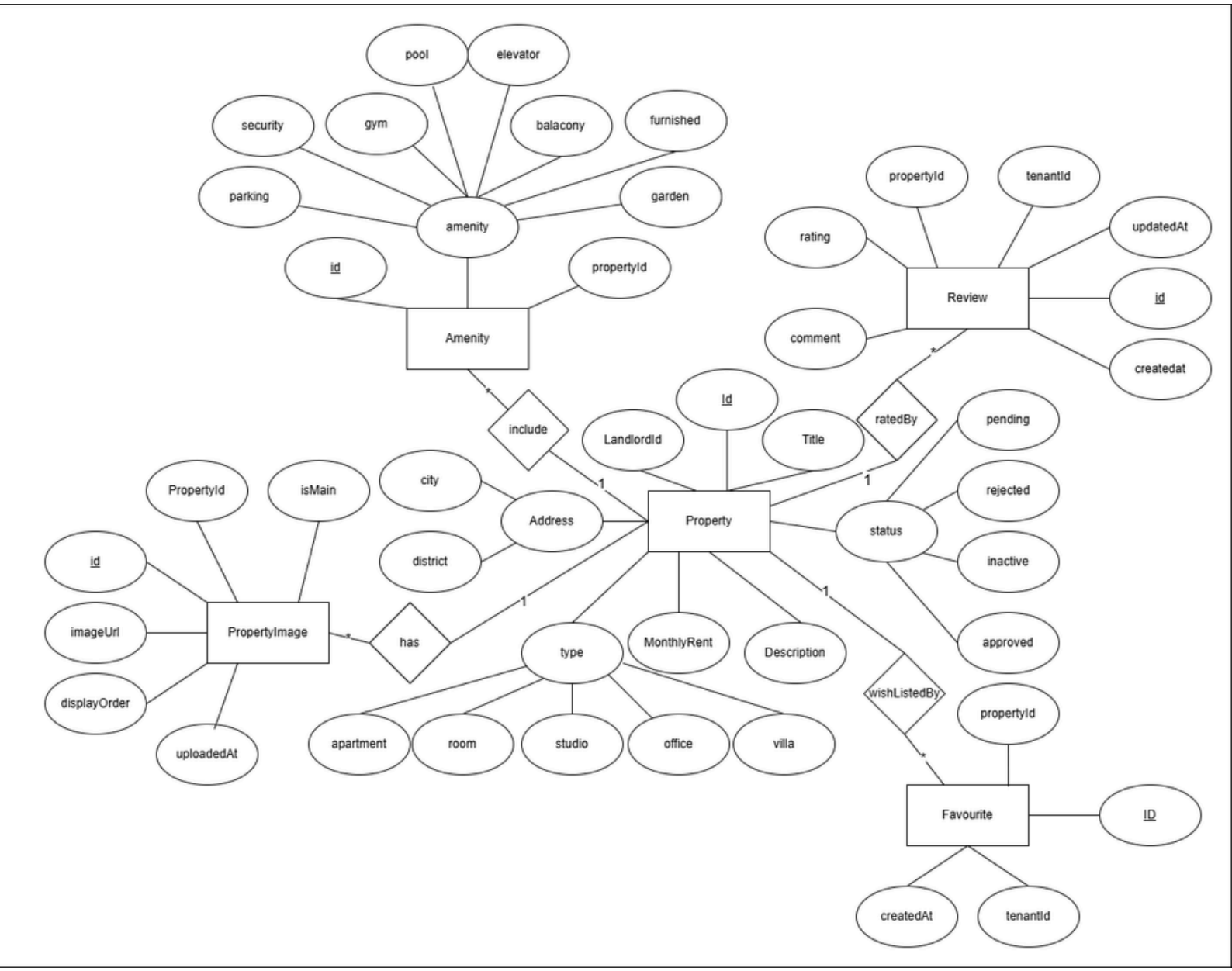
8-ERD (Property Service)

The provided Entity-Relationship Diagram (ERD), titled "Untitled Diagram-ERD-Property.drawio.jpg," illustrates the data schema for the Property Service in the SmartRent platform, centering on the Property entity and its associated components. The Property entity acts as the central hub, containing core attributes such as a unique Id, LandlordId, MonthlyRent, and composite attributes like city (composed of district and address) and type (categorized as apartment, villa, office, etc.).

The diagram defines several key relationships:

- PropertyImage: A property has multiple images (1-to-many relationship), each containing a unique id, imageUrl, and a boolean isMain to identify the primary thumbnail.
- Amenity: Each property can include multiple amenities, which are categorized through a multi-valued attribute covering features like parking, gym, pool, and security.
- Review: Properties are ratedBy tenants (1-to-many relationship), storing a unique id, numerical rating, and a text comment.
- Favourite: A property can be wishListedBy multiple tenants, allowing users to save listings for future reference.

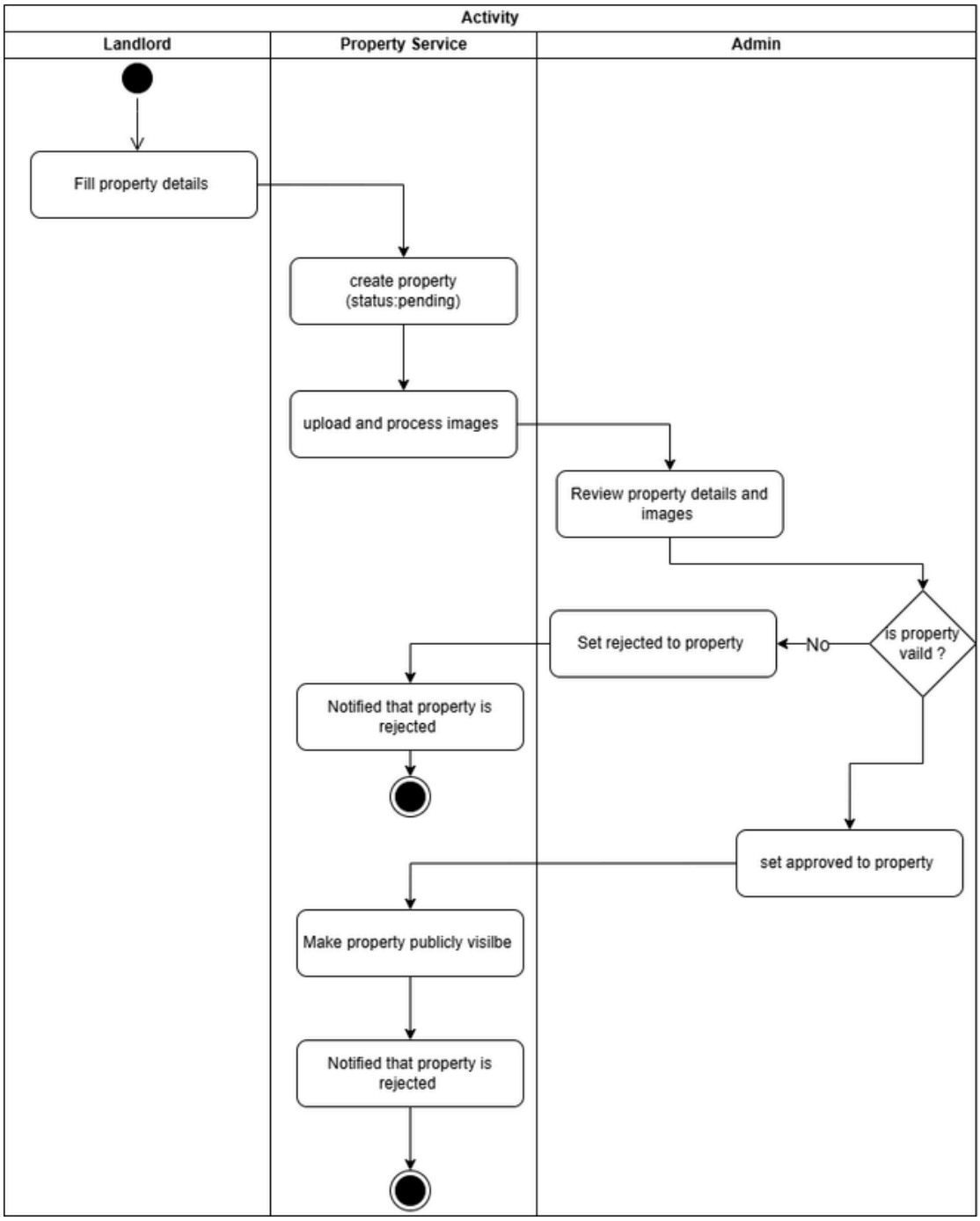
Additionally, the status attribute uses an enumeration to track the lifecycle of a listing through states such as pending, approved, rejected, and inactive, ensuring the platform only displays verified and available content to users.



Diagrams

9-Activity Diagram (Property Service)

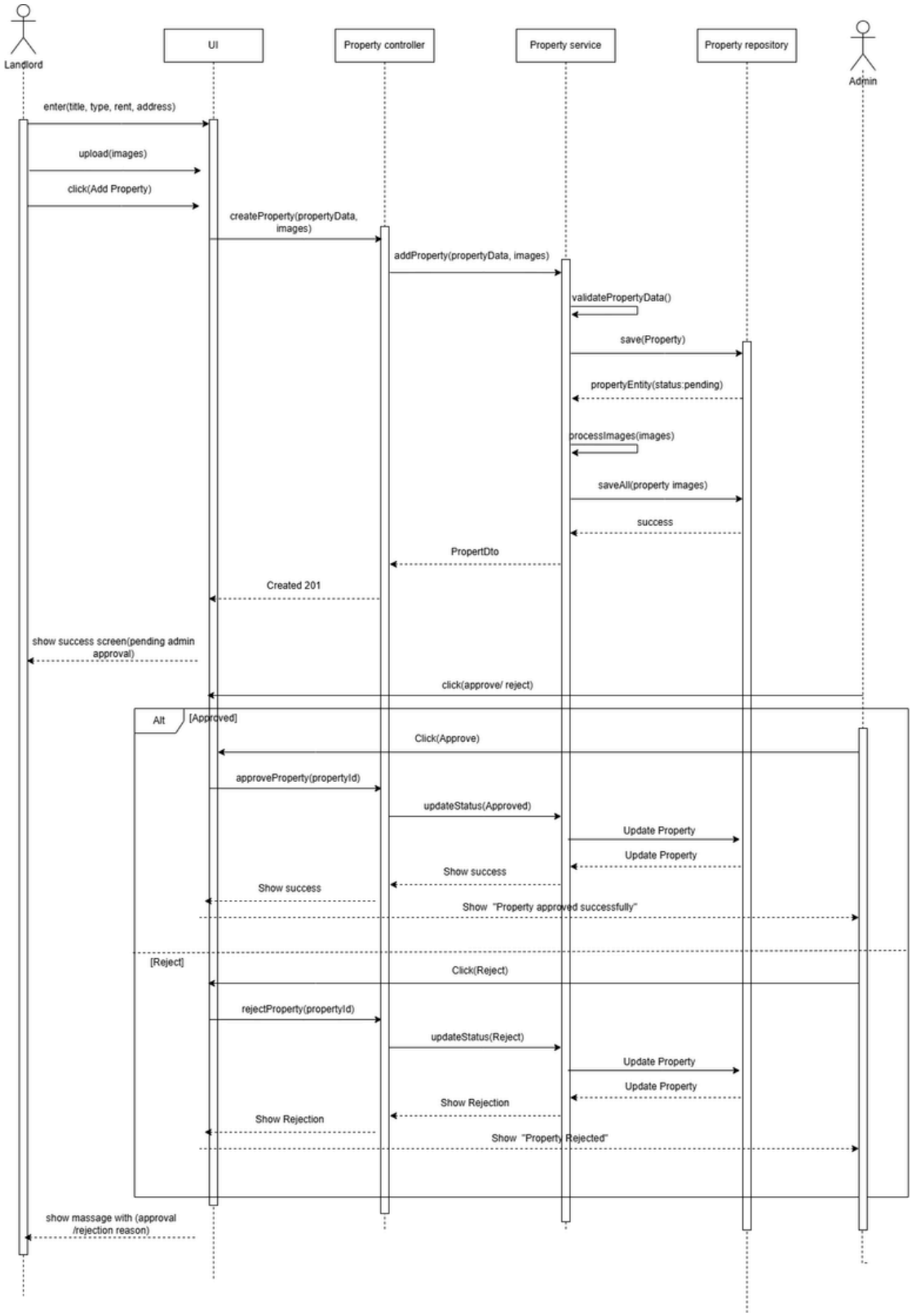
The Activity Diagram for the Property Service outlines the workflow for listing a new real estate asset on the SmartRent platform. The process initiates with the Landlord providing property details, which triggers the Property Service to create a record with a pending status and process uploaded images. Control then shifts to the Admin, who conducts a manual review of the listing's details and visual content. If the property is found to be invalid, the Admin sets the status to rejected, and the system notifies the landlord, ending the process. Conversely, if approved, the Admin updates the status to approved, prompting the system to make the listing publicly visible to tenants. (Note: The final notification box in the provided diagram appears to have a typo, as an approved property would typically trigger a success notification rather than a rejection notice).



Diagrams

10-Sequence Diagram (Property Service)

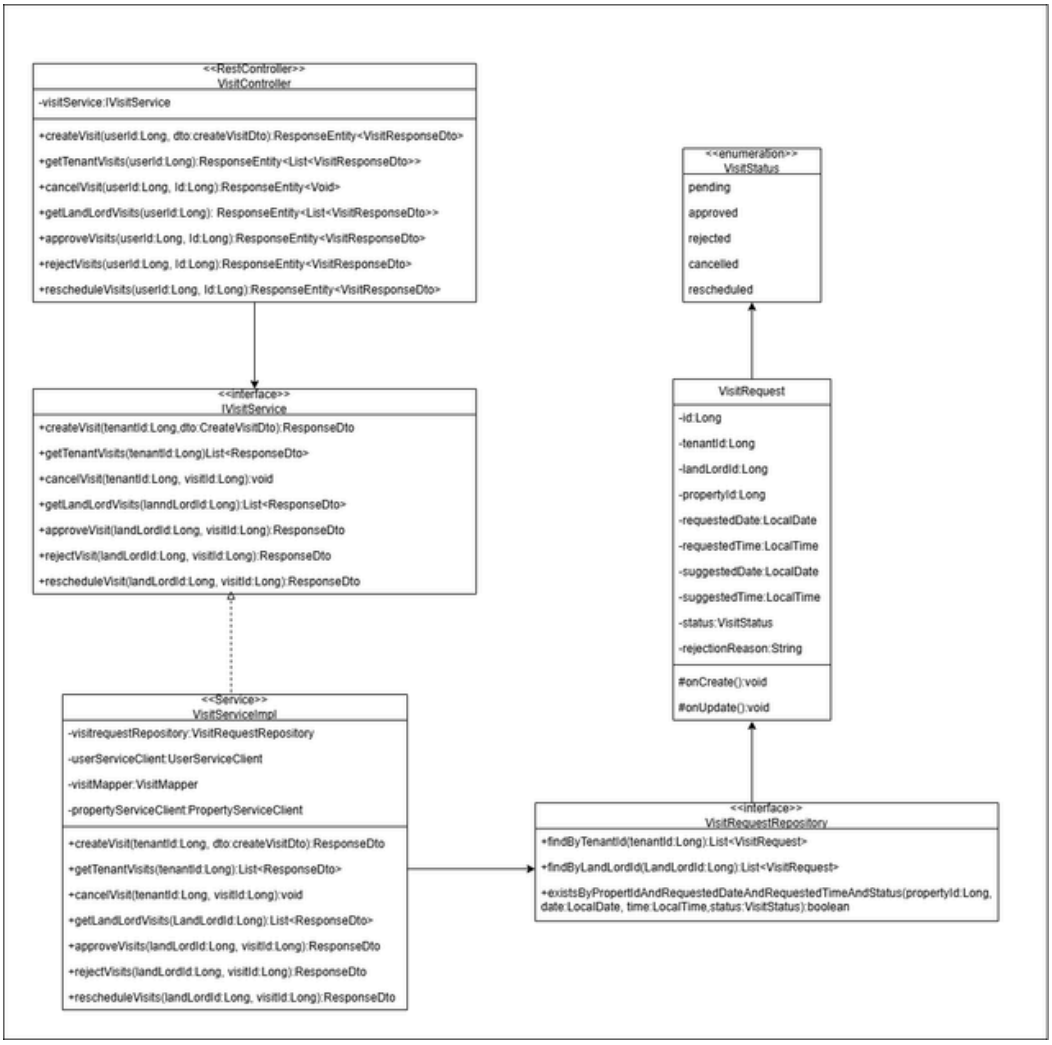
The provided Sequence Diagram illustrates the technical interactions required to list and verify a new real estate asset within the system. The flow begins when a Landlord enters property details and uploads images via the UI, which then calls the Property controller to initiate the creation process. The Property service validates the data, saves the listing with an initial pending status in the Property repository, and processes the associated images. Following this initial submission, the Admin conducts a review, leading to an Alt (alternative) fragment in the logic: if the admin chooses Approve, the service updates the status to Approved in the repository and triggers a success notification; if the admin chooses Reject, the status is updated to Reject. The process concludes by displaying the final approval or rejection message, along with any specific reasons, back to the landlord.



Diagrams

11-Class Diagram (Visit Service)

The provided Class Diagram illustrates the technical architecture of the Visit Service within the SmartRent system. It utilizes a standard layered architecture where the VisitController exposes RESTful endpoints for operations such as creating, approving, rejecting, and rescheduling property visits. This controller communicates with the IVisitService interface, which is implemented by VisitServiceImpl to handle complex business logic, including coordination with external services like the UserServiceClient and PropertyServiceClient via Feign. The data persistence layer is managed by the VisitRequestRepository, which interacts with the VisitRequest entity to store details like tenant and landlord IDs, requested dates, and times. Additionally, a VisitStatus enumeration is used to strictly define the lifecycle of a visit request, ranging from pending and approved to rejected, cancelled, or rescheduled.



OCL For Visit Service class Diagram:

1-context VisitServiceImpl::createVisit(tenantId: Long, dto: CreateVisitDto) : VisitResponseDto
pre NoDoubleBooking:
  visitRequestRepository.existsByPropertyIdAndRequestedDateAndRequestedTimeAndStatus(
    dto.propertyId, dto.requestedDate, dto.requestedTime, VisitStatus::APPROVED
) = false

2-context VisitServiceImpl::createVisit(tenantId: Long, dto: CreateVisitDto) : VisitResponseDto
post InitialStatusIsPending:
  result.status = VisitStatus::PENDING and result.tenantId = tenantId

```
3-context VisitServiceImpl::cancelVisit(tenantId: Long, visitId: Long)
pre MustOwnVisitToCancel:
  let visit : VisitRequest = visitRequestRepository.findById(visitId)->first() in
  visit <> null and visit.tenantId = tenantId
pre CanOnlyCancelPendingOrApproved:
  let visit : VisitRequest = visitRequestRepository.findById(visitId)->first() in
  visit.status = VisitStatus::PENDING or visit.status = VisitStatus::APPROVED
4-context VisitServiceImpl::approveVisit(landlordId: Long, visitId: Long) : VisitResponseDto
pre LandlordMustOwnProperty:
  let visit : VisitRequest = visitRequestRepository.findById(visitId)->first() in
  visit <> null and visit.landlordId = landlordId
5-context VisitServiceImpl::rejectVisit(landlordId: Long, visitId: Long, dto: RejectVisitDto) : VisitResponseDto
pre RejectionReasonIsRequired:
  dto.rejectionReason <> null and dto.rejectionReason.size() > 0
6-context VisitServiceImpl::rejectVisit(landlordId: Long, visitId: Long, dto: RejectVisitDto) : VisitResponseDto
post VisitIsRejected:
  result.status = VisitStatus::REJECTED and result.rejectionReason = dto.rejectionReason
7-context VisitServiceImpl::rescheduleVisit(landlordId: Long, visitId: Long, dto: RescheduleDto) : VisitResponseDto
pre SuggestedDateTimeRequired:
  dto.suggestedDate <> null and dto.suggestedTime <> null
```

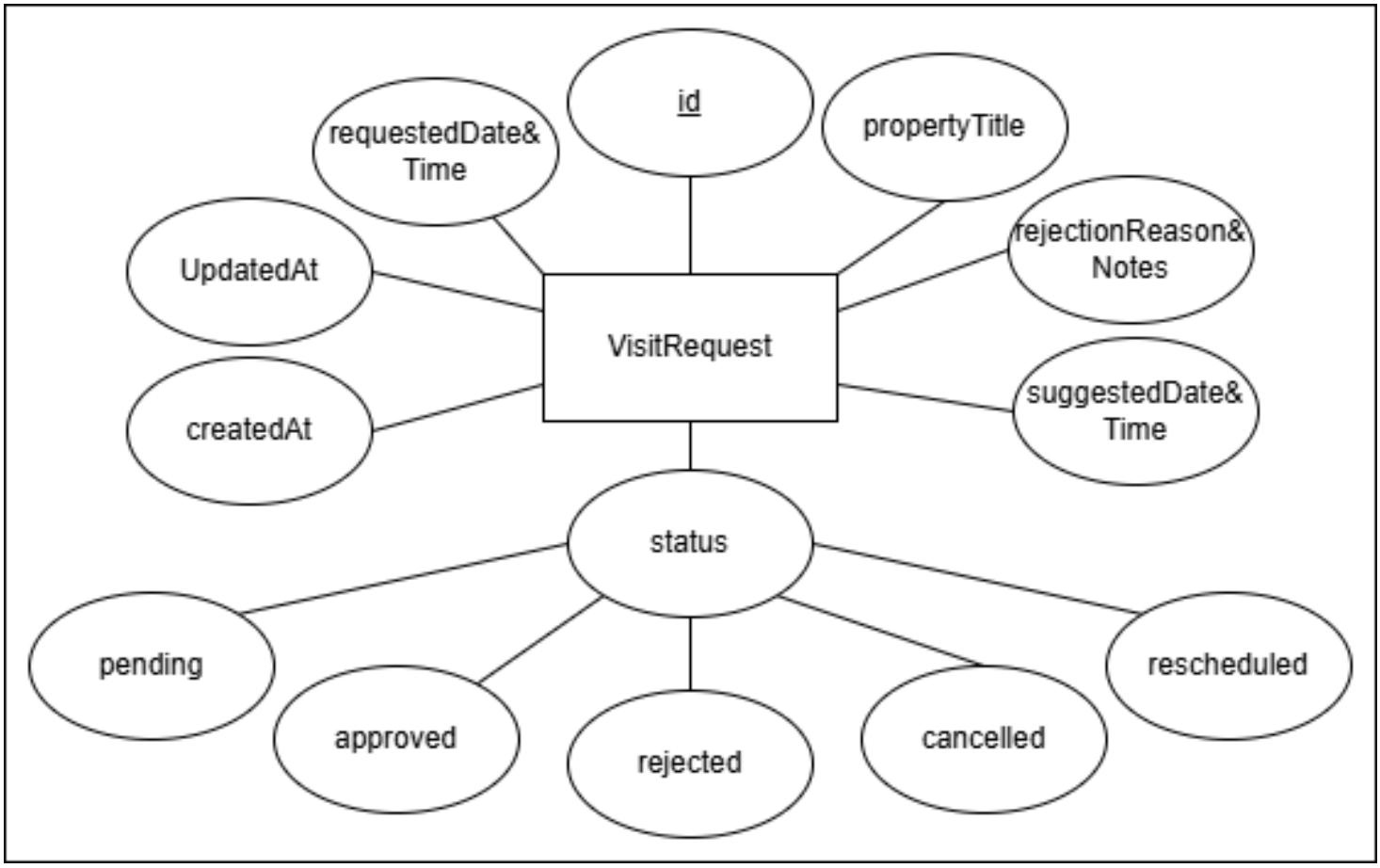
Diagrams

12-ERD (Visit Service)

The provided Entity-Relationship Diagram (ERD), titled "Untitled Diagram-ERD-Visit.drawio.jpg," outlines the data structure for the Visit Service within the SmartRent system. The diagram centers on the VisitRequest entity, which captures all essential data for scheduling property tours between tenants and landlords.

Key components of the diagram include:

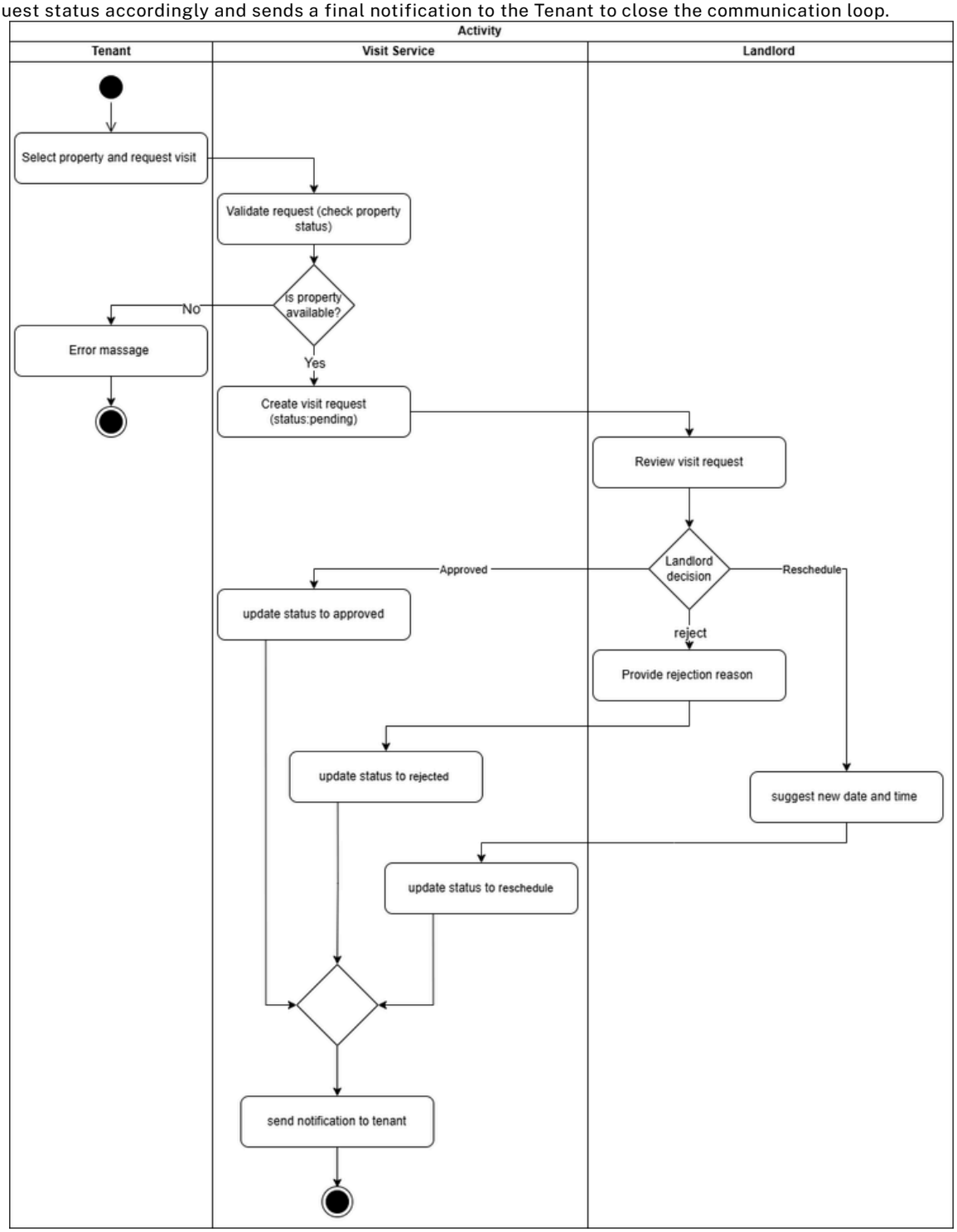
- Core Attributes: The entity is uniquely identified by an id and includes the propertyTitle to maintain a clear reference to the listing in question.
- Temporal Tracking: It records the requestedDate&Time, along with system-generated timestamps for createdAt and updatedAt to track the request's lifecycle.
- Negotiation Fields: The diagram includes specific attributes for rejectionReason&Notes and suggestedDate&Time, which support the system's ability to handle rescheduling and communication when a landlord cannot meet a tenant's initial request.
- Status Management: The status attribute uses an enumeration to strictly manage the state of the visit. These states include pending, approved, rejected, cancelled, and rescheduled, ensuring that both parties have real-time visibility into the progress of the appointment.



Diagrams

13-Activity Diagram (Visit Service)

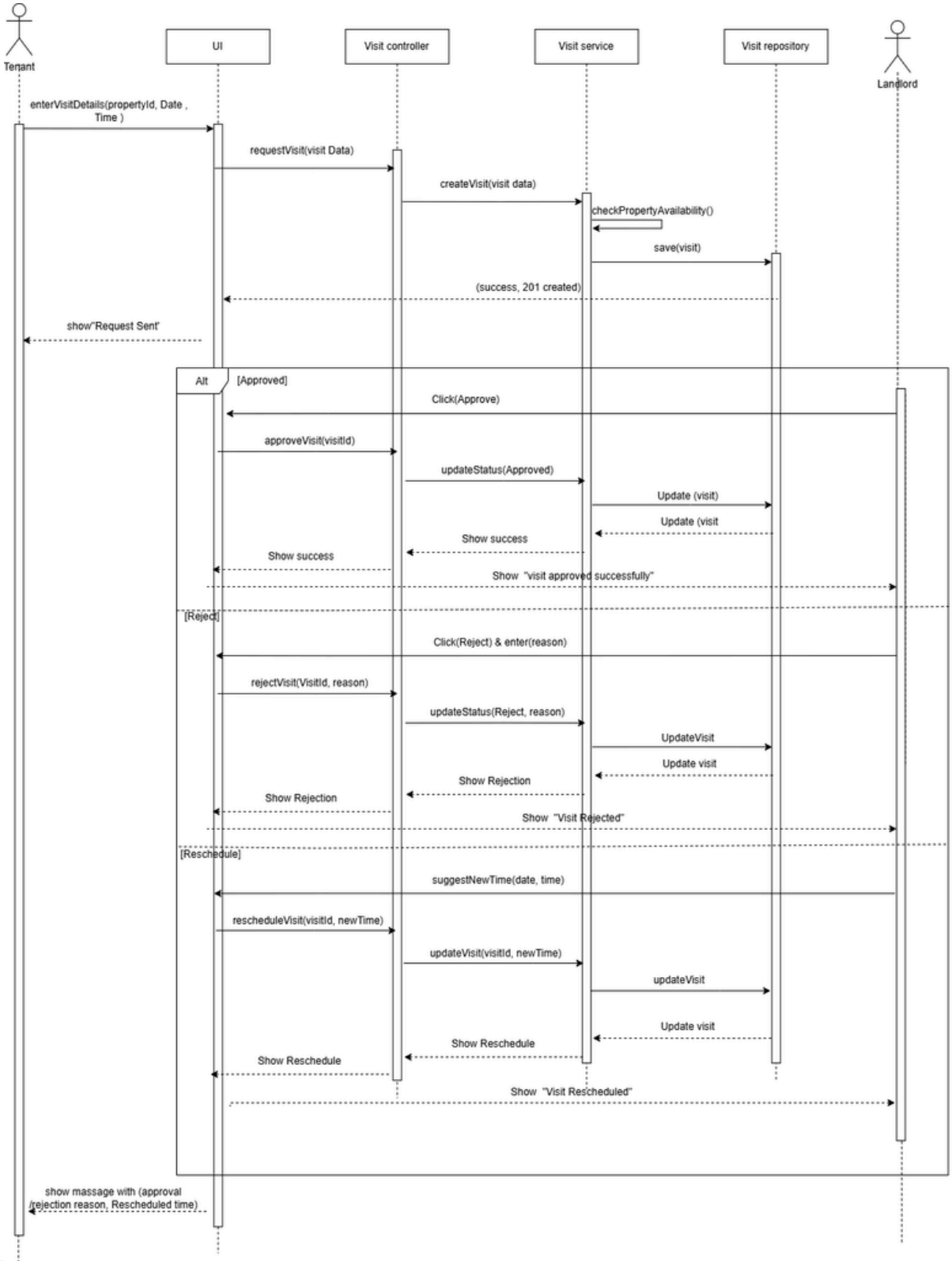
The Activity Diagram for the Visit Service details the dynamic process of coordinating property tours between potential tenants and landlords. The workflow begins with the Tenant selecting a property and requesting a visit, which prompts the Visit Service to validate the property's availability. If the property is unavailable, an error message is returned; otherwise, a visit request is created with a pending status and forwarded to the Landlord. The Landlord then reviews the request and makes a decision that leads to one of three outcomes: approving the visit, rejecting it with a specific reason, or suggesting a reschedule with a new date and time. Once the Landlord's decision is made, the Visit Service updates the request status accordingly and sends a final notification to the Tenant to close the communication loop.



Diagrams

14-Sequence Diagram (Visit Service)

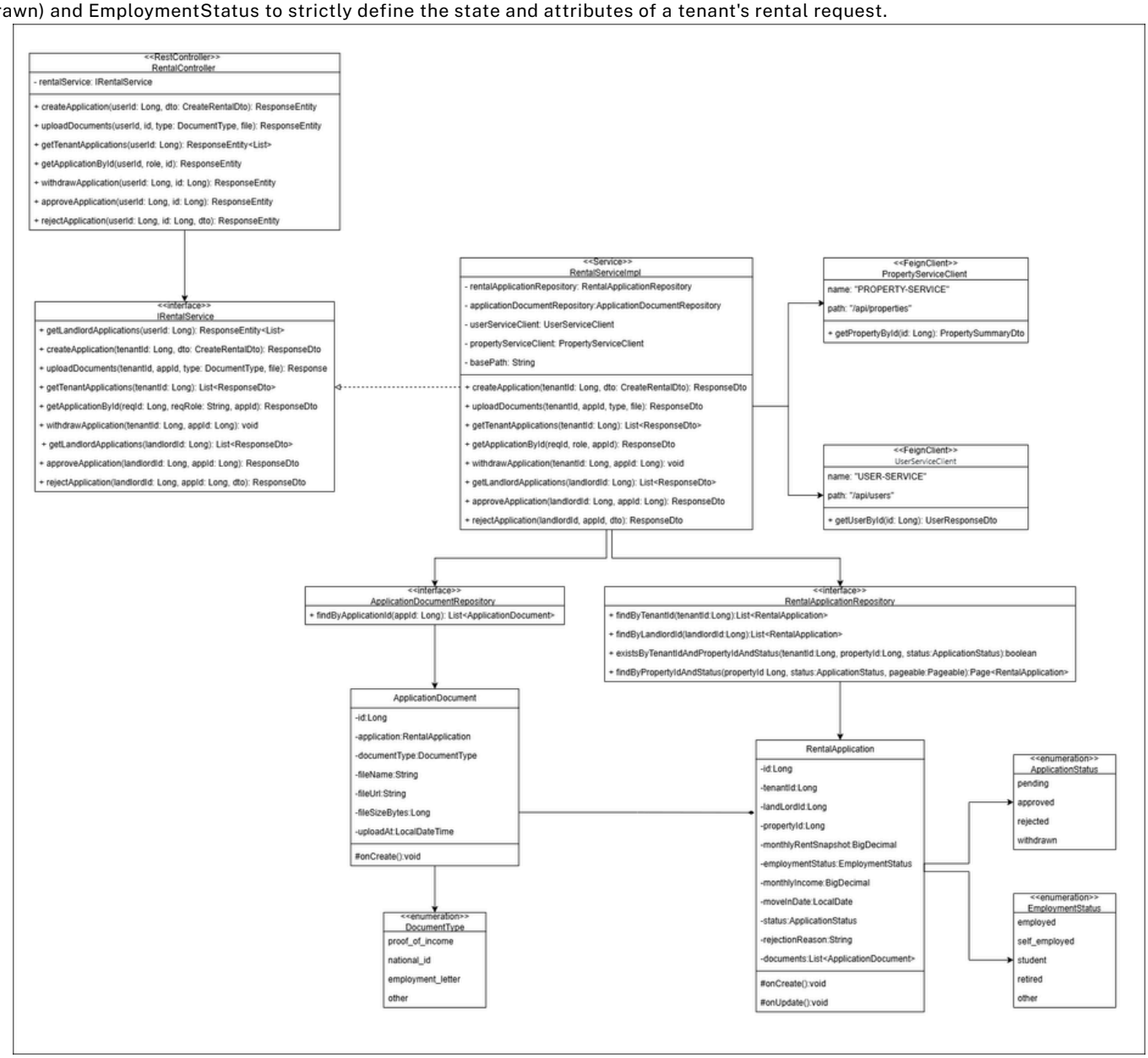
The provided Sequence Diagram for the Visit Service details the technical communication flow between the Tenant, the backend services, and the Landlord. The process is initiated by the Tenant through the UI, which triggers the Visit controller and service to perform a real-time availability check before persisting the request in the Visit repository. The core logic is contained within an "Alt" (Alternative) fragment, which manages the Landlord's response: approving the visit, rejecting it with a specific reason, or rescheduling by suggesting a new time. Each path ensures that the repository is updated and the appropriate success or status notification is returned through the controller and displayed on the UI, maintaining a seamless and transparent coordination loop for all users.



Diagrams

15-Class Diagram (Rental Service)

The provided Class Diagram outlines the technical architecture of the Rental Service within the SmartRent system. It utilizes a layered design where the RentalController exposes endpoints for critical operations such as creating applications, uploading documents, and managing application approvals or rejections. This controller interacts with the IRentalService interface, implemented by RentalServiceImpl, which handles the business logic and coordinates with external microservices via the PropertyServiceClient and UserServiceClient using Feign. The data layer is managed through the RentalApplicationRepository and ApplicationDocumentRepository, which persist the RentalApplication and ApplicationDocument entities. Additionally, the diagram incorporates enumerations like ApplicationStatus (pending, approved, rejected, withdrawn) and EmploymentStatus to strictly define the state and attributes of a tenant's rental request.



OCL For Rental Service class Diagram:

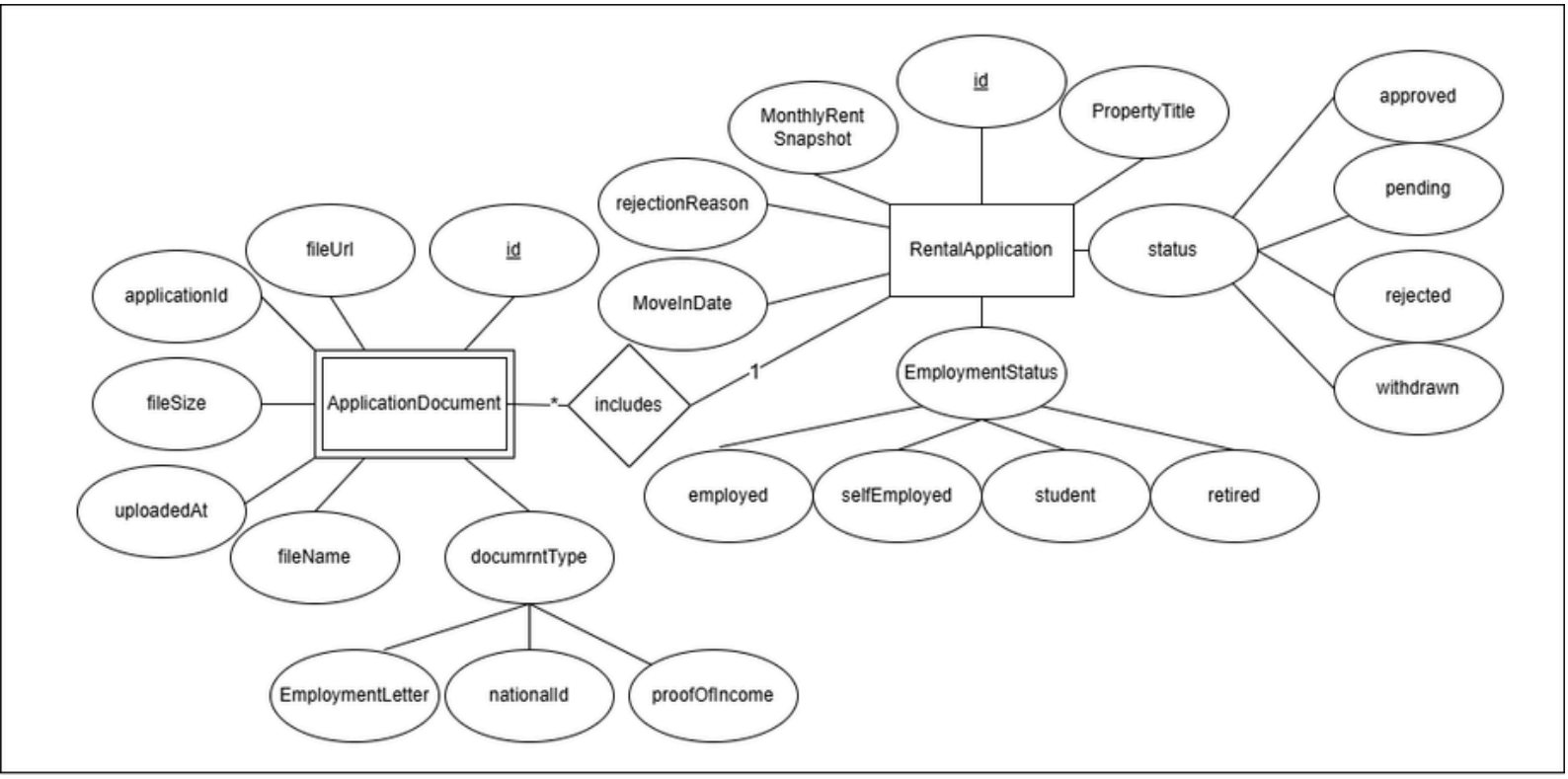
```
1-context RentalServiceImpl::createApplication(tenantId: Long, dto: CreateRentalDto) : RentalResponseDto
pre PropertyIsReady:
  let property = propertyServiceClient.getPropertyById(dto.propertyId) in
  property.status = 'APPROVED' and property.isAvailable = true
pre NoDuplicateApplications:
  rentalApplicationRepository.existsByTenantIdAndPropertyIdAndStatus(tenantId, dto.propertyId, ApplicationStatus::PENDING) = false
  and
  rentalApplicationRepository.existsByTenantIdAndPropertyIdAndStatus(tenantId, dto.propertyId, ApplicationStatus::APPROVED) = false
post InitialStatusSet:
  result.status = ApplicationStatus::PENDING and result.tenantId = tenantId
2-context RentalServiceImpl::uploadDocuments(tenantId: Long, applicationId: Long, type: DocumentType, file: MultipartFile) :
RentalResponseDto
pre UploadAuthorization:
  let application = rentalApplicationRepository.findById(applicationId)->first() in
  application.tenantId = tenantId and application.status = ApplicationStatus::PENDING
pre ValidMimeType:
  file.contentType = 'application/pdf' or file.contentType = 'image/jpeg' or file.contentType = 'image/png'
3-context RentalServiceImpl::withdrawApplication(tenantId: Long, applicationId: Long)
pre WithdrawPrecondition:
  let application = rentalApplicationRepository.findById(applicationId)->first() in
  application.tenantId = tenantId and application.status = ApplicationStatus::PENDING
post StatusUpdatedToWithdrawn:
  rentalApplicationRepository.findById(applicationId)->first().status = ApplicationStatus::WITHDRAWN
4-context RentalServiceImpl::approveApplication(landlordId: Long, applicationId: Long) : RentalResponseDto
pre ApproveAuthorization:
  let application = rentalApplicationRepository.findById(applicationId)->first() in
  application.landlordId = landlordId and application.status = ApplicationStatus::PENDING
post ReviewTimestampUpdated:
  result.status = ApplicationStatus::APPROVED and result.reviewedAt <> null
context RentalServiceImpl::rejectApplication(landlordId: Long, applicationId: Long, dto: RejectRentalDto) : RentalResponseDto
pre RejectAuthorization:
  let application = rentalApplicationRepository.findById(applicationId)->first() in
  application.landlordId = landlordId and application.status = ApplicationStatus::PENDING
pre RejectionReasonIsProvided:
  dto.rejectionReason <> null and dto.rejectionReason.size() > 0

post RejectionFinalized:
  result.status = ApplicationStatus::REJECTED and result.rejectionReason = dto.rejectionReason
```

Diagrams

16-ERD (Rental Service)

The provided Entity-Relationship Diagram (ERD), which represents the data structure for the Rental Service within the SmartRent platform, illustrates the relationship between rental applications and their supporting documentation. The schema centers on the RentalApplication entity, which includes critical attributes such as a unique id, PropertyTitle, MoveInDate, and a MonthlyRentSnapshot to lock in the price at the time of the request. This entity uses two key enumerations: EmploymentStatus (covering employed, self-employed, student, and retired) and status (tracking the application through pending, approved, rejected, and withdrawn states).

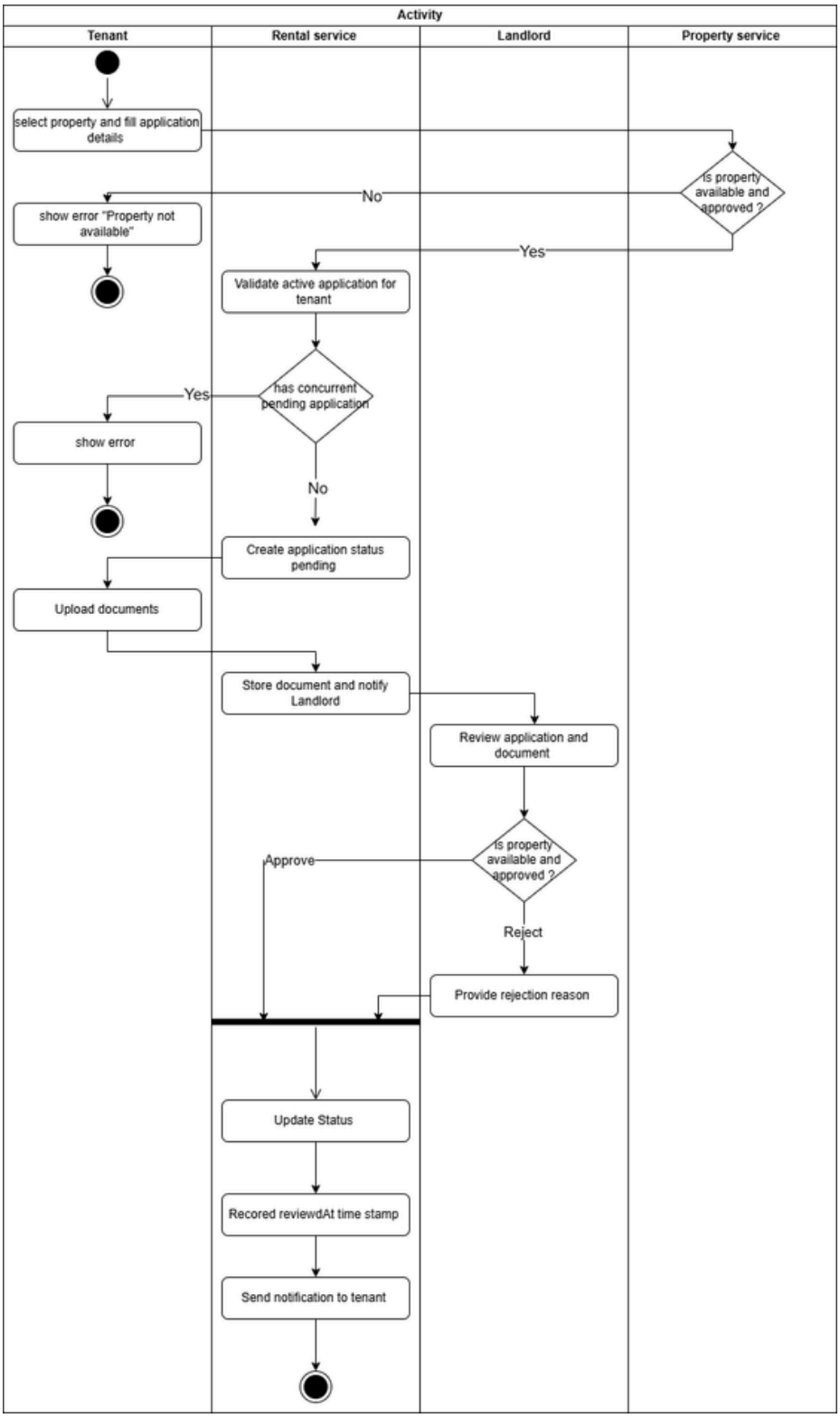
A secondary weak entity, ApplicationDocument, is shown with a double-lined rectangle, indicating it depends on the RentalApplication for its existence. A single RentalApplication can include multiple documents (a 1-to-many relationship), such as an EmploymentLetter, nationalId, or proofOfIncome. Each document entry captures metadata including the fileName, fileUrl, fileSize, and uploadedAt timestamp, ensuring a complete and verifiable digital trail for the landlord's review process.



Diagrams

17-Activity Diagram (Rental Service)

The Activity Diagram for the Rental Service depicts the multi-stage workflow involved in submitting and processing a rental application within the SmartRent platform. The sequence begins with a Tenant selecting a property and providing application details, which triggers a validation check by the Property Service to ensure the listing is currently available and approved. Simultaneously, the Rental Service checks for any concurrent pending applications from the same tenant to prevent duplicate requests. Once these validations pass, the tenant uploads required documentation, and the Landlord is notified to begin their review. The process culminates in the landlord's decision to either approve the application or reject it with a specific reason, after which the system updates the application status, records a "reviewed at" timestamp, and sends a final notification to the tenant.



Diagrams

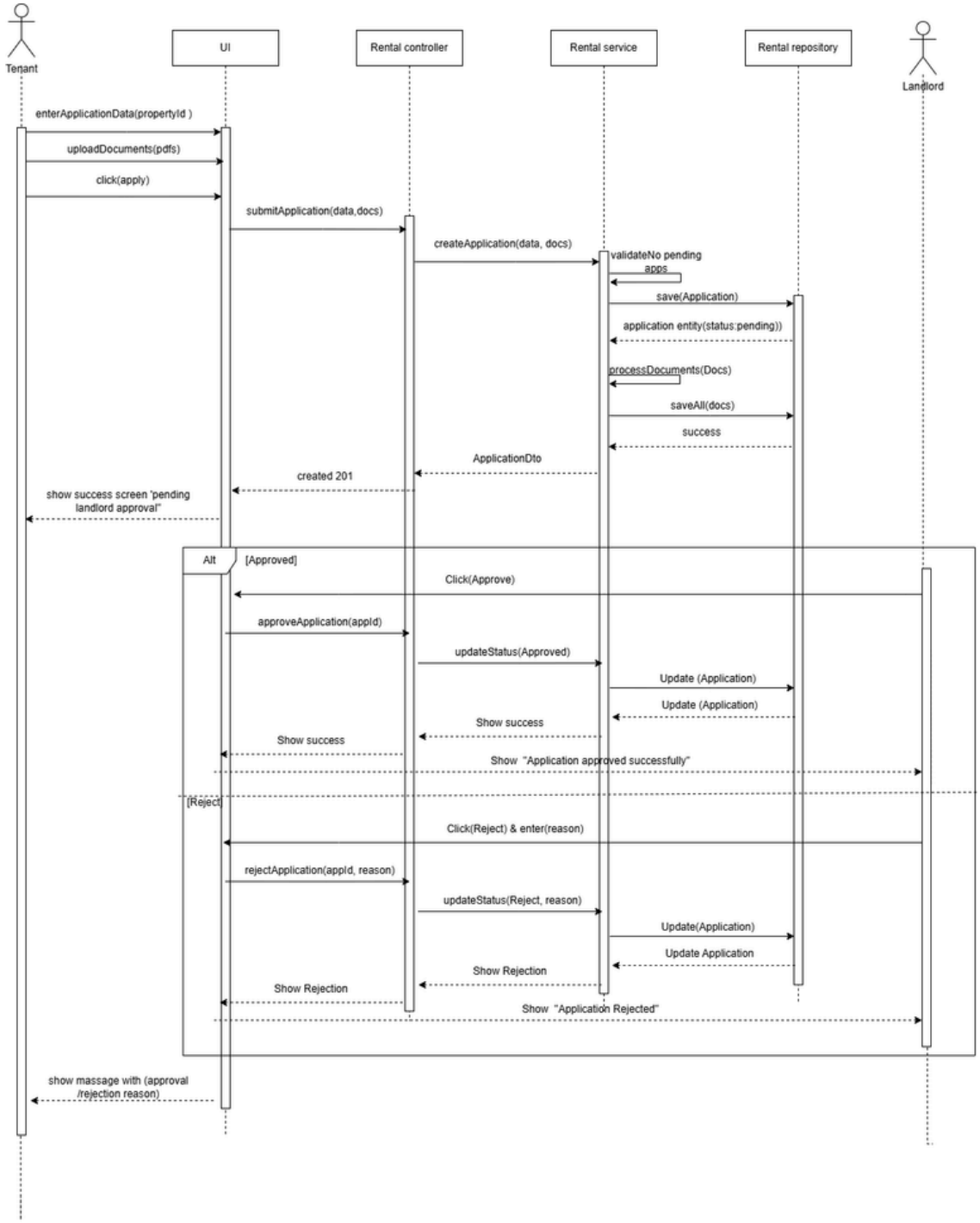
18-Sequence Diagram (Rental Service)

The final Sequence Diagram for the Rental Service outlines the end-to-end technical interaction for lease applications. The process is initiated by the Tenant, who submits property data and uploads necessary documents through the UI. The Rental service then validates that no other pending applications exist for that tenant before saving the application with a pending status and processing the documents in the Rental repository.

Following the submission, the flow enters an "Alt" (Alternative) fragment driven by the Landlord's review:

- Approved: If the landlord clicks approve, the service updates the application status to Approved, persists the change in the repository, and triggers a success notification.
- Reject: If the landlord rejects the application, they must provide a reason; the service then updates the status to Reject and logs the reasoning.

The sequence concludes by relaying the final decision — whether an approval or a rejection with notes — back to the tenant's interface, ensuring a transparent and documented rental process.



The End

Done By:

-Nader Galal	20230620
-Gannah Attia	20230153
-Mariam Hamada	20230538
-Sama Khaled	20230259
-Ali Adel	20230353
-Fatma Maher	20230402
-Ezzat Sobhy	20230346

Thank You :)